
Slprojekt Backend Documentation

Release 0.1

Slprojekt Team

Jan 10, 2026

CONTENTS

1	Overview	1
2	Project Structure	3
3	API Reference	5
3.1	Core	5
3.2	Authentication	5
3.3	Users	6
3.4	Companies	8
3.5	Documents	9
3.6	Internships	10
3.7	Notifications	12
3.8	Services	13
3.9	Shared Utilities	14
4	Building the Docs	15
	Python Module Index	17

OVERVIEW

This documentation covers the backend services, app modules, and build steps for the SIprojekt project. Use the API section for module references and the project structure section for a high-level layout of the repository.

PROJECT STRUCTURE

High-level layout:

```
SIprojekt/  
├── backend/  
│   ├── apps/  
│   ├── core/  
│   ├── services/  
│   └── entrypoint.sh  
├── frontend/  
├── docs/  
├── docker-compose.yml  
└── README.md
```


API REFERENCE

3.1 Core

Root URL configuration for the Django project.

`core.urls.health_check(request)`

Return a simple health check payload for monitoring.

3.2 Authentication

Serializers and validators for OAuth 2.0 endpoints.

`class apps.authentication.oauth_serializers.OAuthAuthorizeSerializer(*args: Any, **kwargs: Any)`

Validate parameters for the authorization code endpoint.

`validate(data)`

`validate_redirect_uri(value)`

`validate_response_type(value)`

`validate_scope(value)`

`class apps.authentication.oauth_serializers.OAuthClientCreateSerializer(*args: Any, **kwargs: Any)`

Validate payload for creating OAuth clients via the API.

`validate(attrs)`

`validate_client_id(value)`

`validate_service_user_id(value)`

`class apps.authentication.oauth_serializers.OAuthTokenSerializer(*args: Any, **kwargs: Any)`

Validate parameters for the token endpoint across grant types.

`validate(data)`

`validate_grant_type(value)`

Simple cache-based rate limiting helpers for OAuth endpoints.

`apps.authentication.rate_limiting.custom_rate_limit(key, limit=10, window=60)`

Return True if the key exceeded the limit within the window.

`apps.authentication.rate_limiting.oauth_rate_limit_check(request, endpoint_type)`

Apply per-client and per-IP limits for OAuth authorize/token endpoints.

3.3 Users

Custom user model and related profiles.

class `apps.users.models.CustomUserManager(*args: Any, **kwargs: Any)`

User manager that enforces roles and password generation.

create_superuser(*email*, *password=None*, ***extra_fields*)

Create a garant superuser with elevated flags.

create_user(*email*, *password=None*, ***extra_fields*)

Create a user with a normalized email and default role.

use_in_migrations = `False`

class `apps.users.models.GarantProfil(*args: Any, **kwargs: Any)`

Garant profile linked to a user record.

class `Meta`

db_table = `'garant_profil'`

managed = `False`

pouzivatel

alias of `User`

class `apps.users.models.StudentProfil(*args: Any, **kwargs: Any)`

Student profile linked to a user record.

class `Meta`

db_table = `'student_profil'`

managed = `False`

pouzivatel

alias of `User`

class `apps.users.models.User(*args: Any, **kwargs: Any)`

Custom user model mapped to existing database columns.

class `Meta`

db_table = `'pouzivatelia'`

managed = `False`

REQUIRED_FIELDS = `['meno', 'priezvisko']`

ROLE_ADMIN = `'admin'`

ROLE_CHOICES = `[('student', 'Študent'), ('garant', 'Garant'), ('firma', 'Firma'), ('externy', 'Externý'), ('admin', 'Admin')]`

```

ROLE_EXTERNY = 'externy'

ROLE_FIRMA = 'firma'

ROLE_GARANT = 'garant'

ROLE_STUDENT = 'student'

USERNAME_FIELD = 'email'

check_password(raw_password)
    Verify a raw password against the stored hash.

has_module_perms(app_label)

has_perm(perm, obj=None)

has_perms(perm_list, obj=None)

property is_active

property is_staff

property is_superuser

property last_login

password = None

set_password(raw_password)
    Hash and store the password in the legacy column.

apps.users.models.validate_student_email(value)
    Validate student email address and allowed domains.

Serializers for user and profile data.

class apps.users.serializers.GarantAccountSerializer(*args: Any, **kwargs: Any)
    Create and serialize garant accounts with password handling.

    class Meta

        fields = ['id', 'email', 'meno', 'priezvisko', 'password', 'pracovisko']

        model
            alias of User

        create(validated_data)

        validate_email(value)

class apps.users.serializers.GarantProfileSerializer(*args: Any, **kwargs: Any)
    Serialize garant profile with user fields.

    class Meta

        fields = ['id', 'meno', 'priezvisko', 'email', 'pracovisko']

        model
            alias of GarantProfil

```

```
class apps.users.serializers.StudentProfileSerializer(*args: Any, **kwargs: Any)
```

Serialize student profile with user fields.

```
class Meta
```

```
    fields = ['id', 'meno', 'priezvisko', 'email', 'studijny_program']
```

```
    model
```

```
        alias of StudentProfil
```

```
class apps.users.serializers.UserSerializer(*args: Any, **kwargs: Any)
```

Serialize core user fields for API responses.

```
class Meta
```

```
    fields = ['id', 'email', 'meno', 'priezvisko', 'is_active', 'is_staff',  
            'is_superuser', 'vytvorene_at']
```

```
    model
```

```
        alias of User
```

Utility helpers for user creation and email delivery.

```
apps.users.utils.generate_strong_password(length: int = 12) → str
```

Generate a strong random password.

```
apps.users.utils.send_garant_credentials_email(user, password: str)
```

Email login credentials to a newly created garant user.

Signals for invalidating cached student data.

```
apps.users.signals.invalidate_student_cache_on_profile_delete(sender, instance, **kwargs)
```

Invalidate cache when a student profile is deleted.

```
apps.users.signals.invalidate_student_cache_on_profile_save(sender, instance, **kwargs)
```

Invalidate cache when a student profile is saved.

```
apps.users.signals.invalidate_student_cache_on_user_delete(sender, instance, **kwargs)
```

Invalidate cache when a student user is deleted.

```
apps.users.signals.invalidate_student_cache_on_user_save(sender, instance, **kwargs)
```

Invalidate cache when a student user is saved.

3.4 Companies

Company database models.

```
class apps.companies.models.Firma(*args: Any, **kwargs: Any)
```

Company record mapped to the existing ‘firmy’ table.

```
class Meta
```

```
    db_table = 'firmy'
```

```
    managed = False
```

Serializers for company data.

```
class apps.companies.serializers.CompanySerializer(*args: Any, **kwargs: Any)
```

Serialize company records.

```
class Meta
```

```
fields = '__all__'
```

```
model
```

alias of [Firma](#)

Signals for invalidating company-related cache entries.

```
apps.companies.signals.invalidate_company_cache_on_delete(sender, instance, **kwargs)
```

Invalidate company cache after delete.

```
apps.companies.signals.invalidate_company_cache_on_save(sender, instance, **kwargs)
```

Invalidate company cache after create/update.

3.5 Documents

```
class apps.documents.models.Dokument(*args: Any, **kwargs: Any)
```

```
class Meta
```

```
db_table = 'dokumenty'
```

```
managed = False
```

```
STAV_CHOICES = [('nahrany', 'Nahrany'), ('potvrdeny', 'Potvrdený'), ('zamietnuty', 'Zamietnutý')]
```

```
STAV_NAHRANY = 'nahrany'
```

```
STAV_POTVRDENY = 'potvrdeny'
```

```
STAV_ZAMIE TNUTY = 'zamietnuty'
```

```
TYP_CHOICES = [('dohoda', 'Dohoda'), ('zmluva', 'Zmluva'), ('zamestnanie', 'Pracovná zmluva'), ('vykaz', 'Výkaz'), ('faktura', 'Faktúra')]
```

```
TYP_DOHODA = 'dohoda'
```

```
TYP_FAKTURA = 'faktura'
```

```
TYP_VYKAZ = 'vykaz'
```

```
TYP_ZAMESTNANIE = 'zamestnanie'
```

```
TYP_ZMLUVA = 'zmluva'
```

```
nahrane_pouzivatel = 'users.User'
```

```
prax = 'internships.Prax'
```

```
skontroloval = 'users.User'
```

```
class apps.documents.serializers.DocumentSerializer(*args: Any, **kwargs: Any)
```

```
class Meta
    fields = '__all__'

    model
        alias of Dokument
```

Company review actions for document approval and rejection.

```
class apps.documents.views.company_review.CompanyReviewMixin
```

Provide company-specific review actions for documents.

```
approve_company(request, pk=None)
    Approve a document as a company and mark it confirmed.

reject_company(request, pk=None)
    Hard-reject a document as a company and mark it rejected.

reject_company_soft(request, pk=None)
    Soft-reject a document as a company without changing state.
```

PDF generation helpers for internship agreement documents.

```
apps.documents.utils.pdf_generator.generate_dohoda_pdf(prax)
    Generate the internship agreement PDF and return (buffer, relative_path).
```

3.6 Internships

Internship models mapped to existing database tables.

```
class apps.internships.models.HistoriaStavovPraxe(*args: Any, **kwargs: Any)
```

History of internship status transitions.

```
class Meta
    db_table = 'historia_stavov_praxe'

    managed = False

    STAV_CHOICES = [('vytvorena', 'Vytvorená'), ('potvrdena', 'Potvrdená'),
                    ('zamietnuta', 'Zamietnutá'), ('schvalena', 'Schválená'), ('obhajena', 'Obhájená'),
                    ('neobhajena', 'Neobhájená')]

    prax
        alias of Prax

    zmenil = 'users.User'
```

```
class apps.internships.models.Prax(*args: Any, **kwargs: Any)
```

Internship record mapped to 'praxe'.

```
FORMA_CHOICES = [('dohoda', 'Dohoda o odbornej praxi'), ('zamestnanie', 'Platené
zamestnanie')]

FORMA_DOHODA = 'dohoda'

FORMA_ZAMESTNANIE = 'zamestnanie'
```

```
class Meta

    db_table = 'praxe'

    managed = False

    SEMESTER_CHOICES = [('zimny', 'Zimný'), ('letny', 'Letný')]

    SEMESTER_LETNY = 'letny'

    SEMESTER_ZIMNY = 'zimny'

    STAV_CHOICES = [('vytvorena', 'Vytvorená'), ('potvrdena', 'Potvrdená'),
                    ('zamietnuta', 'Zamietnutá'), ('schvalena', 'Schválená'), ('obhajena', 'Obhájená'),
                    ('neobhajena', 'Neobhájená')]

    STAV_NEOBHAJENA = 'neobhajena'

    STAV_OBHAJENA = 'obhajena'

    STAV_POTVRDENA = 'potvrdena'

    STAV_SCHVALENA = 'schvalena'

    STAV_VYTVORENA = 'vytvorena'

    STAV_ZAMIETNUTA = 'zamietnuta'

    firma = 'companies.Firma'

    garant = 'users.User'

    student = 'users.User'
```

Permission classes for internship-related views.

```
class apps.internships.permissions.IsCompanyUser(*args: Any, **kwargs: Any)
```

Allow only company role.

```
has_permission(request, view)
```

```
message = 'Prístup je povolený len používateľom s rolou firma.'
```

```
class apps.internships.permissions.IsGarantOrReadOnlyCompany(*args: Any, **kwargs: Any)
```

Garant má plný prístup k firmám, firemný účet len číta vlastnú firmu. Žiadne mutácie pre študentov ani iné roly.

```
has_object_permission(request, view, obj)
```

```
has_permission(request, view)
```

```
message = 'Prístup je povolený len garantovi (plný) alebo firemnému účtu (iba čítanie vlastnej firmy).'
```

```
class apps.internships.permissions.IsGarantOrRelatedDocument(*args: Any, **kwargs: Any)
```

Garant má prístup, inak len účastník praxe, ku ktorej dokument patrí.

```
has_object_permission(request, view, obj)
```

```
has_permission(request, view)
```

```

    message = 'Prístup je povolený len garantovi alebo účastníkovi súvisiacej praxe.'

class apps.internships.permissions.IsGarantOrRelatedInternship(*args: Any, **kwargs: Any)
    Garant má prístup ku všetkému, ostatní len k praxiam, ktorých sú súčasťou.

    has_object_permission(request, view, obj)

    has_permission(request, view)

    message = 'Prístup je povolený len garantovi alebo účastníkovi praxe.'

class apps.internships.permissions.IsGarantUser(*args: Any, **kwargs: Any)
    Ensures only users with the 'garant' role can access the view.

    has_permission(request, view)

    message = 'Prístup je povolený len používateľom s rolou garant.'

class apps.internships.permissions.IsStudentUser(*args: Any, **kwargs: Any)
    Allow only student role.

    has_permission(request, view)

    message = 'Prístup je povolený len používateľom s rolou študent.'
```

Document requirement rules for internship status transitions.

```

apps.internships.document_requirements.missing_required_documents(prax, target_state: str) →
                                                                    List[str]

    Returns a list of human readable document labels that are missing for the requested target state transition.
```

3.7 Notifications

Notification models for email and in-app events.

```

class apps.notifications.models.Notifikacie(*args: Any, **kwargs: Any)
    Notification record used for email delivery and tracking.

    class Meta
        db_table = 'notifikacie'

        STAV_NOVE = 'nove'

        STAV_ODOSLANE = 'odoslane'

        STAV_ZLYHALO = 'zlyhalo'

        prax = 'internships.Prax'

        prijemca = 'users.User'
```

Serializers for notification records.

```

class apps.notifications.serializers.NotificationSerializer(*args: Any, **kwargs: Any)
    Serialize notification records for API responses.

    class Meta
```


`fields = '__all__'`

`model`

alias of *Notifikacie*

Notification service helpers for building and sending messages.

`apps.notifications.service.notify_document_hard_rejected_by_garant`(*document*: *Dokument*, *
(*Keyword-only parameters*
separator (PEP 3102)),
reason: *str*)

Notify student and company about a hard rejection by the garant.

`apps.notifications.service.notify_document_reviewed`(*document*: *Dokument*, *, *approved*: *bool*,
reason: *str* | *None* = *None*)

Notify users about company review outcome for a document.

`apps.notifications.service.notify_document_reviewed_by_garant`(*document*: *Dokument*)

Notify student and company about garant approval.

`apps.notifications.service.notify_document_soft_rejected_by_company`(*document*: *Dokument*, *,
reason: *str*)

Notify a student about a soft rejection by the company.

`apps.notifications.service.notify_document_soft_rejected_by_garant`(*document*: *Dokument*, *,
reason: *str*)

Notify a student about a soft rejection by the garant.

`apps.notifications.service.notify_document_uploaded`(*document*: *Dokument*)

Notify company and garant that a document was uploaded.

Signal handlers for delivering notification emails.

`apps.notifications.signals.send_notification_email`(*sender*, *instance*, *created*, ***kwargs*)

Send email for newly created notification records.

Helper utilities for notification templates and creation.

`apps.notifications.utils.create_notification`(*prax*, *prijemca*, *predmet*, *sablona_kluc*, *payload*=*None*)

Create a notification record; email delivery is handled by signals.

`apps.notifications.utils.get_notification_template`(*stav*, *prax*)

Return (subject, template_key, text) for a given internship state.

3.8 Services

`services.storage.generate_presigned_url`(*object_name*: *str*, *expires_in*: *int* = *None*) → *str*

Generates a temporary (presigned) URL for downloading a file. Only valid for a short time — default from .env.

`services.storage.upload_file_to_b2`(*file_obj*, *filename*: *str* = *None*, *prefix*: *str* = 'uploads/') → *str*

Uploads a file object to Backblaze B2 under given prefix. Vždy uloží kópiu do lokálneho MEDIA_ROOT, aby frontend fungoval cez /media/. Ak je B2 dostupné, uploadne tam rovnaký súbor (best-effort). Returns the *object_name* (relative path) that should be saved in the DB.

3.9 Shared Utilities

Cache key helpers and invalidation utilities.

`apps.cache_utils.build_cache_key(prefix: str, params=None, user=None, extra: str | None = None) → str`

Build a deterministic cache key from prefix, params, and user.

`apps.cache_utils.delete_pattern(pattern: str) → None`

Delete cache entries matching the pattern if supported by backend.

`apps.cache_utils.invalidate_company_cache() → None`

Invalidate caches related to company search and overview.

`apps.cache_utils.invalidate_prax_cache(prax_id=None) → None`

Invalidate caches related to internship lists and details.

`apps.cache_utils.invalidate_student_cache() → None`

Invalidate caches related to student search.

BUILDING THE DOCS

From the project root:

```
sphinx-build -b html docs docs/_build/html
sphinx-build -b latex docs docs/_build/latex
make -C docs/_build/latex latexpdf
```

From the docs directory:

```
sphinx-build -b html . _build/html
sphinx-build -b latex . _build/latex
make -C _build/latex latexpdf
```


PYTHON MODULE INDEX

a

- `apps.authentication.oauth_serializers`, 5
- `apps.authentication.rate_limiting`, 5
- `apps.cache_utils`, 14
- `apps.companies.models`, 8
- `apps.companies.serializers`, 8
- `apps.companies.signals`, 9
- `apps.documents.models`, 9
- `apps.documents.serializers`, 9
- `apps.documents.utils.pdf_generator`, 10
- `apps.documents.views.company_review`, 10
- `apps.internships.document_requirements`, 12
- `apps.internships.models`, 10
- `apps.internships.permissions`, 11
- `apps.notifications.models`, 12
- `apps.notifications.serializers`, 12
- `apps.notifications.service`, 13
- `apps.notifications.signals`, 13
- `apps.notifications.utils`, 13
- `apps.users.models`, 6
- `apps.users.serializers`, 7
- `apps.users.signals`, 8
- `apps.users.utils`, 8

C

- `core.urls`, 5

S

- `services.storage`, 13